7116051
ARYAN

🧠 LangGraph Quiz

📌 Section A: MCQs (1 mark each)

1. What is LangGraph primarily used for?
a) Data visualization
b) Building graph-based AI workflows
c) Database management
d) Web scraping

---

2. In LangGraph, nodes represent:
a) Database tables
b) Functions or steps in a workflow
c) UI components
d) API keys

---

3. What is the main advantage of graph-based workflows?
a) Linear execution only
b) No need for state
c) Flexible branching and looping
d) Faster internet speed

---

4. What does “state” represent in LangGraph?
a) Server location
b) Memory shared across nodes
c) Programming language
d) UI design

---

5. Which component decides the next step in a LangGraph workflow?
a) Tokenizer
b) Edge

c) Prompt
d) Dataset

---

6. A multi-step agent is best described as:
a) A single API call
b) A system executing multiple connected steps
c) A database query
d) A static webpage

---

7. What is the purpose of conditional edges in LangGraph?
a) To delete nodes
b) To control flow based on conditions
c) To store data
d) To train models

---

8. Which of the following is required to define a graph in LangGraph?
a) Nodes and edges
b) Only APIs
c) Only datasets
d) Only UI components

---

9. StateGraph in LangGraph is used for:
a) Styling UI
b) Managing workflow state and transitions
c) Creating databases
d) Sending emails

---

10. What is the output of a node in LangGraph?
a) Only text
b) Updated state
c) Images only
d) HTML code

📌 Section B: Short Answer (2–3 marks each)

11. Explain the concept of graph-based workflows in LangGraph.

Graph-based workflows in LangGraph represent processes as a network of nodes (steps) connected by edges (transitions). Unlike linear workflows, they allow branching, looping, and dynamic execution paths based on conditions.

---

12. What is state management and why is it important in multi-step agents?

State management refers to storing and updating shared data across different steps in a workflow. It is important because multi-step agents rely on previous outputs to make decisions and continue processing correctly.

---

13. Differentiate between nodes and edges in LangGraph.

Nodes are individual functions or steps that perform actions, while edges define the connections and flow between these nodes, deciding how execution moves from one step to another.

---

14. What is a StateGraph and how does it help in building agents?

A StateGraph is a structure in LangGraph that manages both the workflow steps and the shared state. It helps in building agents by controlling how data flows and how transitions occur between steps.

---

15. Give one real-world example where a multi-step agent can be useful.

A customer support chatbot that understands a query, categorizes it (billing, technical issue), and then provides the appropriate response.

📌 Section C: Coding-Based Questions (4–5 marks each)

16. Consider the following code:

```
def step1(state):
```

```
    return {"msg": "Hello " + state["name"]}
```

👉 What is happening in this function? Explain.

The function step1 takes a state dictionary as input and returns a new dictionary with a message. It concatenates "Hello " with the value of state["name"], effectively generating a greeting using the user's name.

---

17. Write a simple LangGraph workflow with two steps:

- Step 1: Take user input
- Step 2: Generate a response

```
define step1(state):
    state["input"] = get_user_input()
    return state

define step2(state):
    state["response"] = "You said: " + state["input"]
    return state

graph = StateGraph()

graph.add_node("input_step", step1)
graph.add_node("response_step", step2)

graph.add_edge("input_step", "response_step")

graph.set_entry_point("input_step")
```

---

18. How would you add a conditional step in a LangGraph workflow? Explain with logic.

To add a conditional step, you define a function that checks a condition and routes to different nodes:

Logic Example:

```
def decision(state):
    if "exam" in state["query"]:
        return "exam_node"
    else:
        return "assignment_node"
```

```
graph.add_conditional_edges("decision_node",decision)
```

📌 Section D: Case Study (5–8 marks)

19. Design a multi-step agent using LangGraph for a “Student Assistant Bot” that:

- Takes a question
- Checks if it is about assignments or exams
- Routes to the correct handler
- Returns a response

👉 Explain:

- Nodes : The bot consists of several nodes representing different steps. The Input Node collects the student’s question, the Decision Node determines whether the query is about assignments or exams, and based on that, it routes to either the Assignment Handler Node or the Exam Handler Node. Finally, the Response Node generates and returns the answer.
- Edges : Edges define how the workflow moves between nodes. The flow starts from the Input Node to the Decision Node. From there, conditional edges route the query to either the Assignment Handler or Exam Handler. After processing, both handlers connect to the Response Node to deliver the final output.
- State : The state stores important information throughout the workflow. It keeps the user’s question, identifies the category (assignment or exam), and stores the generated response, allowing smooth data flow across all steps.

📌 Section E: Advanced Conceptual (5 marks)

20. Compare traditional linear AI pipelines with graph-based agent workflows. Why is LangGraph better for complex systems?

Traditional linear AI pipelines follow a fixed sequence of steps, meaning they cannot easily adapt to different scenarios or loop back. In contrast, graph-based workflows allow flexible paths, conditional branching, and iterative loops.

LangGraph is better for complex systems because:

- It supports dynamic decision-making
- Enables reusable components (nodes)
- Handles multi-step reasoning efficiently
- Maintains shared state across the workflow